

Option C Task Report

COS30018 – Intelligent Systems

Lecturer: Dr. Nguyen Manh Toan

Group: 3.1 – OpenBuffet

Task name: **C.5 – Machine Learning 2**



When it comes to the machine learning section of this week's assignment, which is called C5: Machine Learning 2, then the LSTM/GRU/RNN algorithm that was created in the previous section (C.4) was expanded in order to forecast two more complicated tasks, such as multistep prediction (forecasting closing prices in k steps in the future instead of forecasting the next day's price) and multivariate prediction (using the following prices as an input: Open, High, Low, Close and Volume instead of only Close price). Consequently, multivariate multistep predictor has been developed. In all the cases above, the same generalised functions are applied, therefore the feature/ k can be included/excluded arbitrarily.

Here are the detailed code descriptions of our work:

1. Task_5/data_loader.py / prepare_sequences()

```

145 def prepare_sequences(data: np.ndarray, target_idx: int,
146                        prediction_days: int, k_steps: int = 1):
147     """
148     Build supervised windows for the multivariate and/or multistep problems (Task C.5).
149
150     This generalises prepare_lstm_data() along two axes at once:
151     * multivariate : 'data' may have several feature columns, all of which go
152       into each input window.
153     * multistep   : each target is the next 'k_steps' values of the target
154       column (not just the single next value).
155
156     Parameters
157     -----
158     data      : 2-D array of shape (n_rows, n_features) of SCALED values.
159     target_idx : Column index of the feature we predict (e.g. Close).
160     prediction_days : Look-back window length (timesteps fed to the model).
161     k_steps    : How many future steps to predict (k >= 1).
162
163     Returns
  
```

PROBLEMS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

```

def prepare_sequences(data: np.ndarray, target_idx: int,
                      k_steps: int) -> tuple:
    """
    Returns
    -----
    X : np.ndarray (n_samples, prediction_days, n_features) - input windows.
    y : np.ndarray (n_samples, k_steps) - next k target values.

    Note: with a single feature column and k_steps=1 this reduces to the same
    pairs as prepare_lstm_data (only y is 2-D here, shape (n_samples, 1)).
    """
    if data.ndim != 2:
        raise ValueError("data must be 2-D (n_rows, n_features).")
    if k_steps < 1:
        raise ValueError("k_steps must be >= 1.")

    X, y = [], []
    # Stop k_steps early so every window has a full k-length target ahead of it.
    for i in range(prediction_days, len(data) - k_steps + 1):
        X.append(data[i - prediction_days:i, :]) # look-back, all features
        y.append(data[i + k_steps, target_idx]) # next k target values
    return np.array(X), np.array(y)

```

PROBLEMS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

`prepare_multivariate_multistep_lstm_data()` is the main part of the processing in Task C.5. This function differs from the previous `prepare_lstm_data()` function in two ways: it can take more than one feature column in the input window (multivariate) and the target output of each window will be k -steps long.

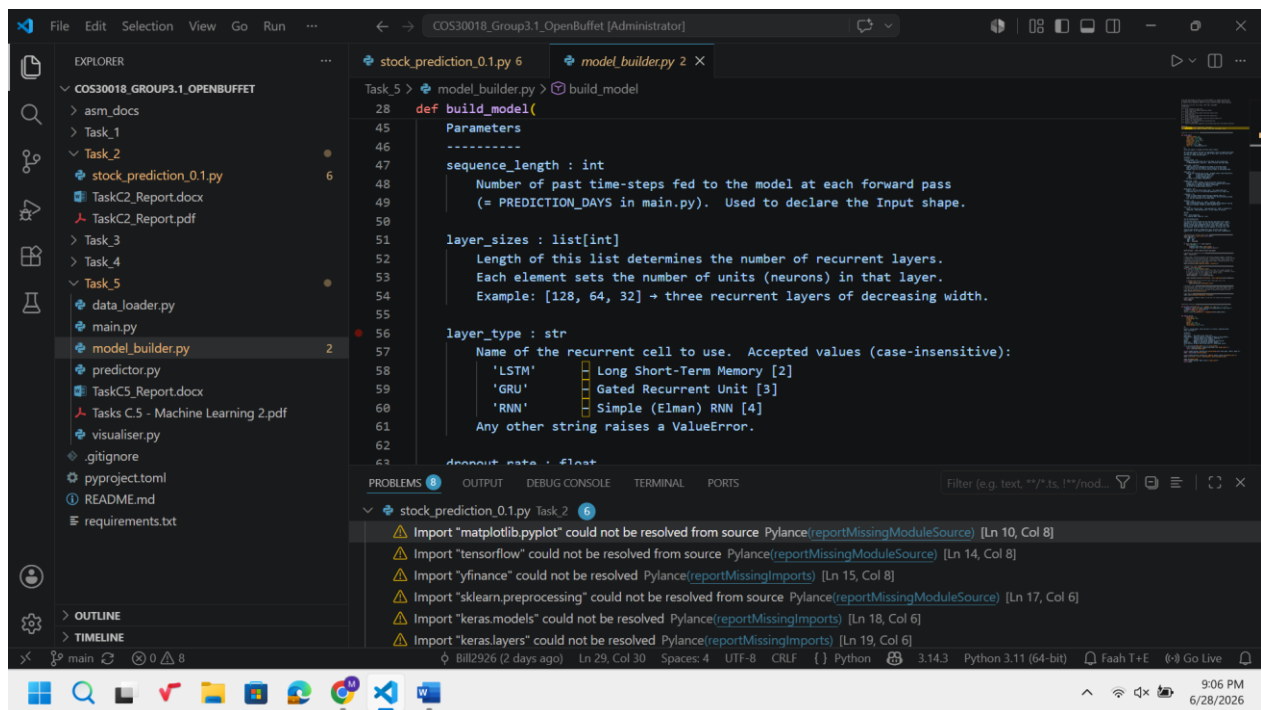
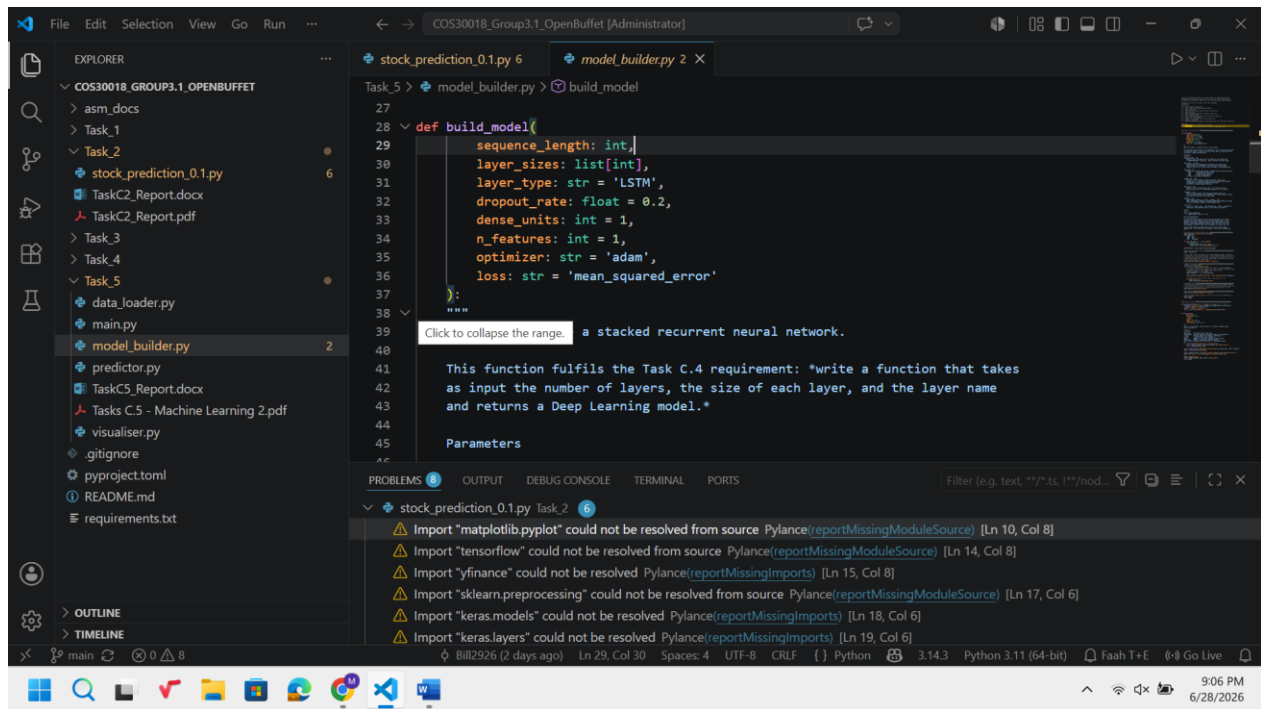
As one can see, the end of the loop range is now `len(data) - k_steps + 1` and not just `len(data)`. If the change is not done, the last windows will go out of the range while trying to generate the target vector and thus, `IndexError` will be raised or the target vector may become partially filled. The $+1$ is used because of exclusive nature of Python slicing range, so the last window should be able to produce a target vector of length k .

In the loop, `data[i - prediction_days : i, :]` uses 2-D slicing of NumPy arrays and selects all columns (`:`) of the desired rows. In this way, the multivariate property of the function is ensured: using of the data array with 5 columns (Open, High, Low, Close, Volume) gives input windows of shape `(prediction_days, 5)` without any additional conditionals.

`data[i : i + k_steps, target_idx]` selects only the target column (for example, the third column for Close) of the next k rows. Selection of only the target column among other input columns guarantees that the model will know about all features, but predict only one of them — typical multivariate to univariate forecasting problem.

One can see that the shape of `y` is `(n_samples, k_steps)`. When `k_steps = 1`, the `y` becomes a 2-D column vector, not 1-D, as returned by `prepare_lstm_data()`. The decision was made on purpose to ensure 2-D shape of the target. Such target shape is required for Keras Dense layer with `dense_units = k_steps`.

2. Task_5/model_builder.py / build_model() — n_features & dense_units



File Edit Selection View Go Run ... COS30018_Group3.1_OpenBuffet [Administrator]

EXPLORER

- COS30018_Group3.1_OPENBUFFET
 - asm_docs
 - Task_1
 - Task_2
 - stock_prediction_0.1.py
 - TaskC2_Report.docx
 - TaskC2_Report.pdf
 - Task_3
 - Task_4
 - Task_5
 - data_loader.py
 - main.py
 - model_builder.py
 - predictor.py
 - TaskC5_Report.docx
 - Tasks C.5 - Machine Learning 2.pdf
 - visualiser.py
 - .gitignore
 - pyproject.toml
 - README.md
 - requirements.txt

Task_5 > model_builder.py > build_model

```

28 def build_model(
63     dropout_rate : float
64     Fraction of units randomly zeroed during each training step.
65     Dropout [5] is a widely-used regularisation technique that prevents
66     co-adaptation of neurons and reduces overfitting.
67     Set to 0.0 to disable dropout entirely.
68
69     dense_units : int
70     Width of the final Dense output layer. For single-step price
71     prediction this is 1; for multistep prediction it is k (Task C.5).
72
73     n_features : int
74     Number of input feature time-series per timestep (Task C.5).
75     1 for univariate (Close only); len(OHLCV) for the multivariate problem.
76     Sets the last dimension of the Input shape.
77
78     optimizer : str
79     Keras optimiser name, e.g. 'adam', 'rmsprop', 'sgd'.
80     Adam is the default because it combines momentum and adaptive learning
81     rates and converges reliably on most sequence tasks.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*, !**/mod...)

- stock_prediction_0.1.py Task 2
 - Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
 - Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
 - Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
 - Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
 - Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
 - Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

main Bill2926 (2 days ago) Ln 29, Col 30 Spaces: 4 UTF-8 CRLF Python 3.14.3 Python 3.11 (64-bit) Faah T+E Go Live

9:07 PM 6/28/2026

File Edit Selection View Go Run ... COS30018_Group3.1_OpenBuffet [Administrator]

EXPLORER

- COS30018_Group3.1_OPENBUFFET
 - asm_docs
 - Task_1
 - Task_2
 - stock_prediction_0.1.py
 - TaskC2_Report.docx
 - TaskC2_Report.pdf
 - Task_3
 - Task_4
 - Task_5
 - data_loader.py
 - main.py
 - model_builder.py
 - predictor.py
 - TaskC5_Report.docx
 - Tasks C.5 - Machine Learning 2.pdf
 - visualiser.py
 - .gitignore
 - pyproject.toml
 - README.md
 - requirements.txt

Task_5 > model_builder.py > build_model

```

28 def build_model(
84
85     loss : str
86     Keras loss function name. 'mean_squared_error' (MSE) is standard for
87     regression; 'mean_absolute_error' is more robust to outliers.
88
89     Returns
90     -----
91     model : keras.Sequential
92     A compiled model ready for .fit().
93
94     How the stacking works
95     -----
96     All recurrent layers except the last one must pass their *full output
97     sequence* to the next layer, not just the final hidden state. This is
98     done by setting return_sequences=True on every layer except the last.
99     Without it the shape collapses to (batch, units) after the first layer
100    and subsequent recurrent layers have no sequence to iterate over. [6]

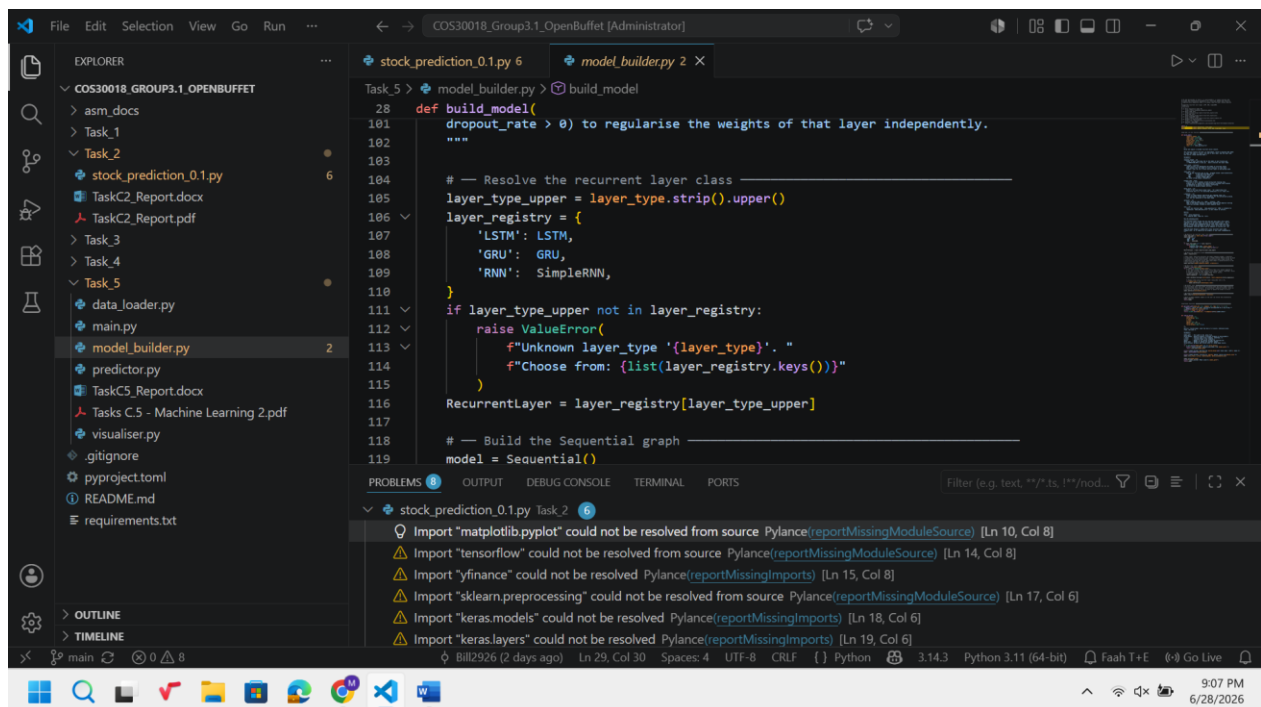
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*, !**/mod...)

- stock_prediction_0.1.py Task 2
 - Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
 - Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
 - Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
 - Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
 - Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
 - Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

main Bill2926 (2 days ago) Ln 29, Col 30 Spaces: 4 UTF-8 CRLF Python 3.14.3 Python 3.11 (64-bit) Faah T+E Go Live

9:07 PM 6/28/2026



In Task C.5 the `build_model()` function created in Task C.4 is utilized without any changes in the code by passing two additional keyword arguments – `n_features` and `dense_units`. Any changes in the code structure are not necessary since those two parameters were added to the arguments' list.

The `n_features` parameter is responsible for the setting the last dimension of `Input(shape=(sequence_length, n_features))` layer. For the univariate baseline case `n_features = 1` as each timestep contains only one value. For the multivariate case `n_features = 5` (Open, High, Low, Close, Volume), which means that the model gets matrixes as timesteps instead of vector. This particular dimensionality of the input is automatically propagated in all recurrent layers by the Keras framework.

The `dense_units` parameter defines the number of units in the output layer: `model.add(Dense(units=dense_units))`. In the case of single-step prediction (Task C.4) `dense_units = 1`, which means that the output of the model is one price. In the case of multistep prediction (Task C.5) `dense_units = k`, which means that the output of the model is `k` prices during one forward pass. It is an example of the direct multi-output strategy where instead of predicting sequentially each timestep we predict all `k` timesteps in one forward pass using one lookback window.

3. Task_5/predictor.py / forecast_future()

```

def forecast_future(model, last_window: np.ndarray, close_scaler: MinMaxScaler,
                    k_steps: int) -> np.ndarray:
    """
    Forecast the next `k_steps` closing prices in one shot (Task C.5 multistep).

    A model built with dense_units=k outputs all k future values from a single
    look-back window, so we just feed the most recent window and invert scaling.

    Parameters
    -----
    last_window : scaled array (prediction_days, n_features) - the latest window.
    close_scaler: scaler fitted on the Close column (for inverse-transform).
    k_steps     : number of future days predicted (must match the model output).

    Returns
    -----
    np.ndarray of shape (k_steps,) - predicted closing prices in USD.
    """
    pred_days, n_features = last_window.shape

```

Problems (7) OUTPUT DEBUG CONSOLE TERMINAL PORTS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

```

def forecast_future(model, last_window: np.ndarray, close_scaler: MinMaxScaler,
                    look_back_window, so we just feed the most recent window and invert scaling.

    Parameters
    -----
    last_window : scaled array (prediction_days, n_features) - the latest window.
    close_scaler: scaler fitted on the Close column (for inverse-transform).
    k_steps     : number of future days predicted (must match the model output).

    Returns
    -----
    np.ndarray of shape (k_steps,) - predicted closing prices in USD.
    """
    pred_days, n_features = last_window.shape
    window = last_window.reshape(1, pred_days, n_features)
    scaled_pred = model.predict(window, verbose=0) # shape (1, k_steps)
    return inverse_close(scaled_pred, close_scaler).flatten()

```

Problems (7) OUTPUT DEBUG CONSOLE TERMINAL PORTS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

It deals with the prediction part through inference. It accepts the latest lookback window and produces k predicted price points in regular USD.

Window = last_window.reshape(1, pred_days, n_features) generates an additional batch dimension with index 0. The keras model expects the input always in the format (batch_size, timesteps, features). In the case of training, the batch size dimension is handled automatically by keras; but in the case of prediction, while feeding a single lookback window to model.predict(), it

needs to be handled manually through either reshape or `np.expand_dims()`. If not done, keras will receive a 2D input and throw an error on mismatch of shape.

The output from keras model is of the shape (1, k_steps), i.e., one sample of k predicted scaled price points. This is further processed with `inverse_close()`, where the output is reshaped into a column vector and the data is inverse scaled using Close scaler. There are two reasons behind keeping `inverse_close()` outside of the two functions: one, avoiding code repetition.

4. Task_5/predictor.py / build_test_inputs_mv() & make_windows()

The screenshot shows the VS Code editor with the file `predictor.py` open. The function `build_test_inputs_mv` is defined as follows:

```

84 def build_test_inputs_mv(
85     train_df: pd.DataFrame,
86     test_df: pd.DataFrame,
87     feature_cols: list[str],
88     scalers: dict,
89     prediction_days: int
90 ) -> np.ndarray:
91     """
92     Multivariate version of build_test_inputs().
93
94     Prepend the last `prediction_days` rows of the training period to the test
95     period (so the first test day has a full look-back window), then scale every
96     feature column with its OWN fitted scaler (from data_loader's `scalers` dict,
97     all fitted on training data only - no leakage).
98
99     Returns a 2-D array (len(test)+prediction_days, n_features) of scaled values,
100     with columns in the order of `feature_cols`.
101     """
102     combined = pd.concat([train_df[feature_cols], test_df[feature_cols]], axis=0)

```

The Problems panel at the bottom shows several import errors:

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

```

84 def build_test_inputs_mv(
85     # Multivariate version of build_test_inputs().
86     train_df: pd.DataFrame,
87     test_df: pd.DataFrame,
88     prediction_days: int,
89 ) -> np.ndarray:
90     """
91     Prepend the last 'prediction_days' rows of the training period to the test
92     period (so the first test day has a full look-back window), then scale every
93     feature column with its OWN fitted scaler (from data_loader's 'scalers' dict,
94     all fitted on training data only - no leakage).
95
96     Returns a 2-D array (len(test)+prediction_days, n_features) of scaled values,
97     with columns in the order of 'feature_cols'.
98     """
99     combined = pd.concat([train_df[feature_cols], test_df[feature_cols]], axis=0)
100     tail = combined.iloc[len(combined) - len(test_df) - prediction_days:]
101
102     scaled = np.empty(tail.shape, dtype=float)
103     for j, col in enumerate(feature_cols):
104         # transform (never fit) so the test data uses the training statistics.
105         scaled[:, j] = scalers[col].transform(tail[[col]].values).flatten()
106     return scaled

```

PROBLEMS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

```

107     # transform (never fit) so the test data uses the training statistics.
108     scaled[:, j] = scalers[col].transform(tail[[col]].values).flatten()
109     return scaled
110
111
112 def make_windows(model_inputs: np.ndarray, prediction_days: int) -> np.ndarray:
113     """
114     Multivariate version of make_predictions(): slide a look-back window over a
115     2-D scaled array but keep ALL feature columns in each window.
116
117     Returns x_test of shape (n_windows, prediction_days, n_features).
118     """
119     x_test = []
120     for i in range(prediction_days, len(model_inputs)):
121         x_test.append(model_inputs[i - prediction_days:i, :]) # all features
122     return np.array(x_test)

```

PROBLEMS

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

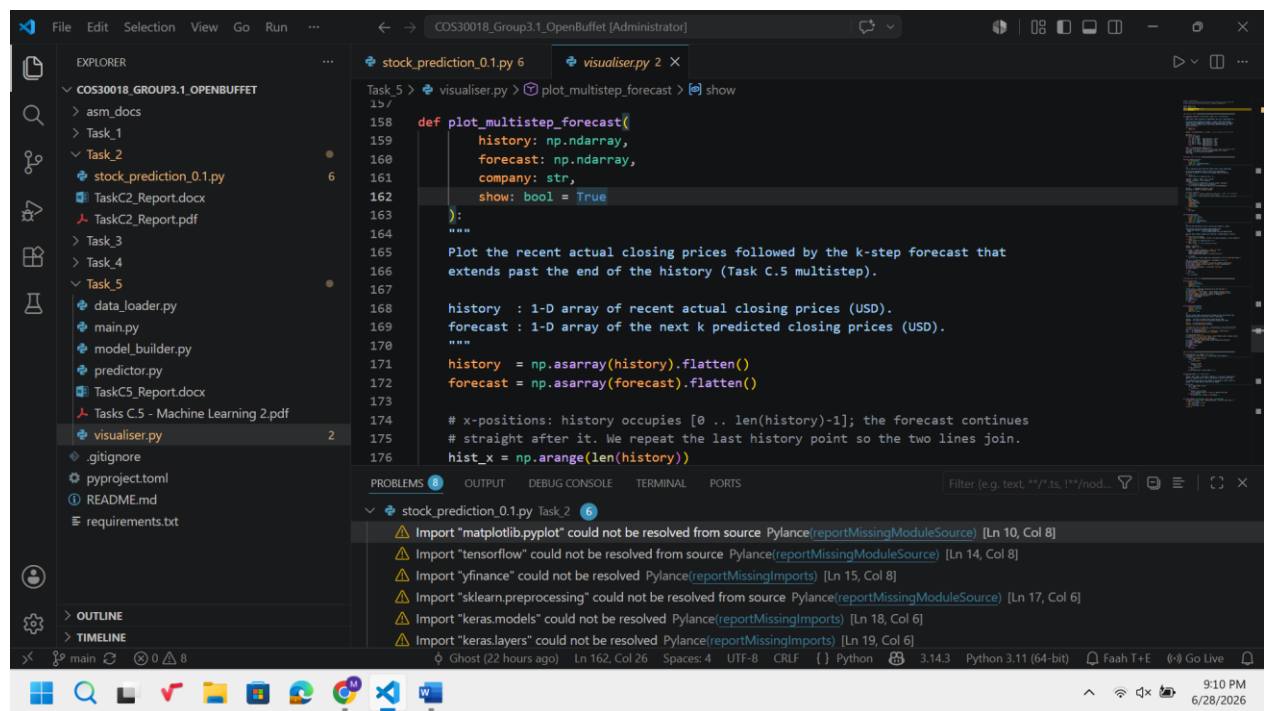
The multivariate version of the function `build_test_inputs()` is `build_test_inputs_mv()`. In this function, we add the last `prediction_days` days of the training period into the test period so that the first day of the test period contains the complete look-back window similar to the univariate version, but this time it will be done on all the feature columns.

The important line in this function is: `scaled[:, j] = scalers[col].transform(tail[[col]].values).flatten()`. The reason why this line individual normalization of the columns is performed using their respective scalers because Open, High,

Low, Close, and Volume have completely different ranges of values (\$10s for prices, and \$100 million in case of Volume). Had we performed one scaler for all these columns, then the values of all other columns except one column would have become either zero or one. We lose the relative variations in the columns which we need in the multivariate input. Since we have fitted the scalers only on the training data (data_loader.py), we use transform instead of fit_transform here.

The function make_windows() is same as the function make_prediction() except that the part model_inputs[i - prediction_days : i, :] is replaced by model_inputs[i - prediction_days : i, :]. Now we extract windows of size (prediction_days, n_features) instead of (prediction_days, 1). The shape of the output is (n_windows, prediction_days, n_features).

5. Task_5/visualiser.py / plot_multistep_forecast()



```

Task_5 > visualiser.py > plot_multistep_forecast > show
158 def plot_multistep_forecast(
159     # x-positions: history occupies [0 .. len(history)-1]; the forecast continues
160     # straight after it. We repeat the last history point so the two lines join.
161     hist_x = np.arange(len(history))
162     fc_x = np.arange(len(history) - 1, len(history) + len(forecast))
163     fc_y = np.concatenate([history[-1:], forecast])
164
165     plt.figure(figsize=(12, 5))
166     plt.plot(hist_x, history, color='black', label=f'Recent actual {company} Close')
167     plt.plot(fc_x, fc_y, color='red', marker='o',
168             label=f'{len(forecast)}-day forecast')
169     plt.title(f'{company} - {len(forecast)}-Day Closing Price Forecast')
170     plt.xlabel('Trading Days')
171     plt.ylabel('Price (USD)')
172     plt.legend()
173     plt.tight_layout()
174     if show:
175         plt.show()
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, "/*", "*/", "mod...")

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

main 0 0

9:10 PM 6/28/2026

The plot represents the past prices plotted together with the extension of the graph via k-step forecast.

The x-axis values are calculated as follows: $fc_x = np.arange(len(history) - 1, len(history) + len(forecast))$. We should start from the index which is one before the last value of the historical data ($len(history) - 1$) to make sure that the forecast line will begin from the same point from which the actual line ends to ensure the continuity of the two lines. If we take $fc_x = np.arange(len(history), len(history) + len(forecast))$, then there will be one-step discontinuity between the lines.

The y-values of the forecast line can be represented by the following formula: $fc_y = np.concatenate([history[-1:], forecast])$. This is because the forecast line begins from $history[-1:]$, i.e., the last value of history due to the specific of $np.concatenate$ function.

6. Task_5/main.py — Three Prediction Scenarios

```

Task 5 > main.py > ...
206 recent_close = train_data[PRICE_VALUE].values[-PREDICTION_DAYS:] # raw, for plots
207
208
209 # -- [C.5-1] Multistep: univariate Close + next k closes -----
210 X1, y1 = prepare_sequences(train_close, target_idx=0,
211                             prediction_days=PREDICTION_DAYS, k_steps=K_STEPS)
212 model_multistep = load_or_build(
213     model_path = get_model_path(MODEL_DIR, COMPANY, tag=f'multistep_k_{K_STEPS}'),
214     build_kwargs = dict(sequence_length=PREDICTION_DAYS, layer_sizes=LAYER_SIZES,
215                         layer_type=DL_NETWORK, dropout_rate=DROPOUT_RATE,
216                         dense_units=K_STEPS, n_features=1),
217     x_train=X1, y_train=y1, epochs=EPOCHS, batch_size=BATCH_SIZE,
218     force_retrain=FORCE_RETRAIN,
219 )
220 forecast_1 = forecast_future(model_multistep, train_close[-PREDICTION_DAYS:],
221                             close_scaler, K_STEPS)
222 print(f"[C.5-1] Multistep (K_STEPS)-day Close forecast (USD): {np.round(forecast_1, 2)}")
223 plot_multistep_forecast(recent_close, forecast_1, COMPANY)
224
225

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, **/*, !**/mod...)

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

main 0 0 7

9:15 PM 6/28/2026

```

Task 5 > main.py > ...
226 # -- [C.5-2] Multivariate: OHLCV + next-day Close -----
227 X2, y2 = prepare_sequences(train_features, target_idx=target_idx,
228                             prediction_days=PREDICTION_DAYS, k_steps=1)
229 model_multivariate = load_or_build(
230     model_path = get_model_path(MODEL_DIR, COMPANY, tag='multivariate'),
231     build_kwargs = dict(sequence_length=PREDICTION_DAYS, layer_sizes=LAYER_SIZES,
232                         layer_type=DL_NETWORK, dropout_rate=DROPOUT_RATE,
233                         dense_units=1, n_features=len(FEATURES)),
234     x_train=X2, y_train=y2, epochs=EPOCHS, batch_size=BATCH_SIZE,
235     force_retrain=FORCE_RETRAIN,
236 )
237 # Evaluate across the whole test period using multivariate look-back windows.
238 mv_inputs = build_test_inputs_mv(train_data, test_data, FEATURES, scalers, PREDICTION_DAYS)
239 x_test_mv = make_windows(mv_inputs, PREDICTION_DAYS)
240 predicted_mv = inverse_close(model_multivariate.predict(x_test_mv), close_scaler)
241 plot_predictions(test_data[TARGET_COL].values, predicted_mv, COMPANY)
242
243

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, **/*, !**/mod...)

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

main 0 0 7

9:15 PM 6/28/2026

```

243
244 # -- [C.5-3] Combined: multivariate OHLCV + next k closes
245 X3, y3 = prepare_sequences(train_features, target_idx=target_idx,
246                             prediction_days=PREDICTION_DAYS, k_steps=K_STEPS)
247 model_combined = load_or_build(
248     model_path = get_model_path(MODEL_DIR, COMPANY, tag=f'mv_multistep_k{K_STEPS}'),
249     build_kwargs = dict(sequence_length=PREDICTION_DAYS, layer_sizes=LAYER_SIZES,
250                         layer_type=DL_NETWORK, dropout_rate=DROPOUT_RATE,
251                         dense_units=K_STEPS, n_features=len(FEATURES)),
252     x_train=X3, y_train=y3, epochs=EPOCHS, batch_size=BATCH_SIZE,
253     force_retrain=FORCE_RETRAIN,
254 )
255 forecast_3 = forecast_future(model_combined, train_features[-PREDICTION_DAYS:],
256                             close_scaler, K_STEPS)
257 print(f"[C.5-3] Multivariate {K_STEPS}-day Close forecast (USD): {np.round(forecast_3, 2)}")
258 plot_multistep_forecast(recent_close, forecast_3, COMPANY)
259

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, "/*", "*/", "mod...")

- Import "matplotlib.pyplot" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 10, Col 8]
- Import "tensorflow" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 14, Col 8]
- Import "yfinance" could not be resolved Pylance(reportMissingImports) [Ln 15, Col 8]
- Import "sklearn.preprocessing" could not be resolved from source Pylance(reportMissingModuleSource) [Ln 17, Col 6]
- Import "keras.models" could not be resolved Pylance(reportMissingImports) [Ln 18, Col 6]
- Import "keras.layers" could not be resolved Pylance(reportMissingImports) [Ln 19, Col 6]

Scenario C.5 uses the same two functions for all three scenarios, namely `prepare_sequences()` and `build_model()`, however, changes their arguments.

[C.5-1] Multistep – univariate: `prepare_sequences(train_close, target_idx=0, k_steps=K_STEPS)` is used in the case of the one-column dataset (shape $(n, 1)$), where `target_idx=0` (one feature) and `k_steps=K_STEPS` is the number of steps that is defined by a user. The model will be trained on `n_features=1` and `dense_units=K_STEPS`, that is, it will learn to transform the last `K_STEPS` values of Close to the next `K_STEPS` values of Close.

[C.5-2] Multivariate – single step: `prepare_sequences(train_features, target_idx=3, k_steps=1)` applies for the five-column dataset (shape $(n, 5)$), where `target_idx=3` refers to Close and `k_steps=1`. The model will be trained on `n_features=5` and `dense_units=1`, which means that the model will have more information about the features and will forecast the Close of the next day.

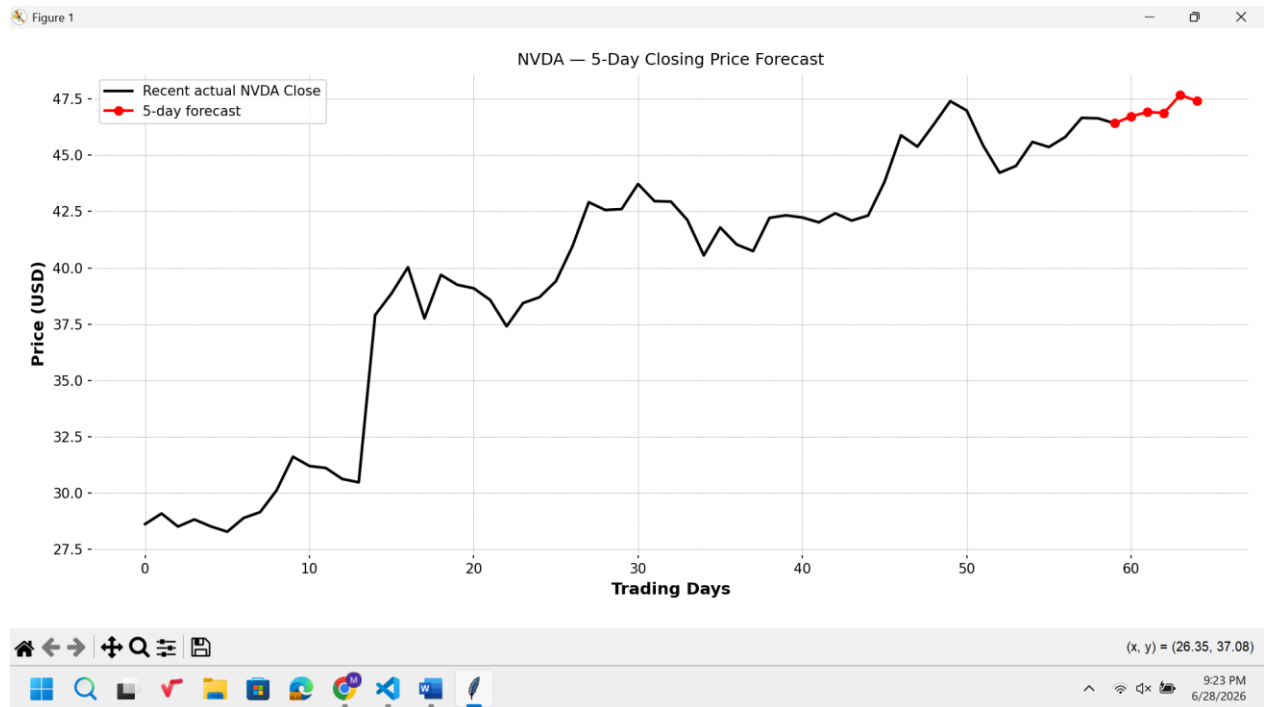
[C.5-3] Multivariate and multistep: the same features and `k_steps=K_STEPS` are used, however, `dense_units=K_STEPS`. Thus, the scenario is the most generalised one: five features will be required for forecasting the value of Close `K_STEPS` days ahead. The difference between the scenario C.5-2 and C.5-3 is only `dense_units`, the rest is the same data pipeline.

The `ask_int("Forecast horizon k ...", default=5)` lets a user define `K_STEPS` interactively. It will be better not to use the try/except block in the `main.py` in order to avoid the incorrect passing of the argument (0 or negative) to the model and use the helper function.

7. Experiment Results

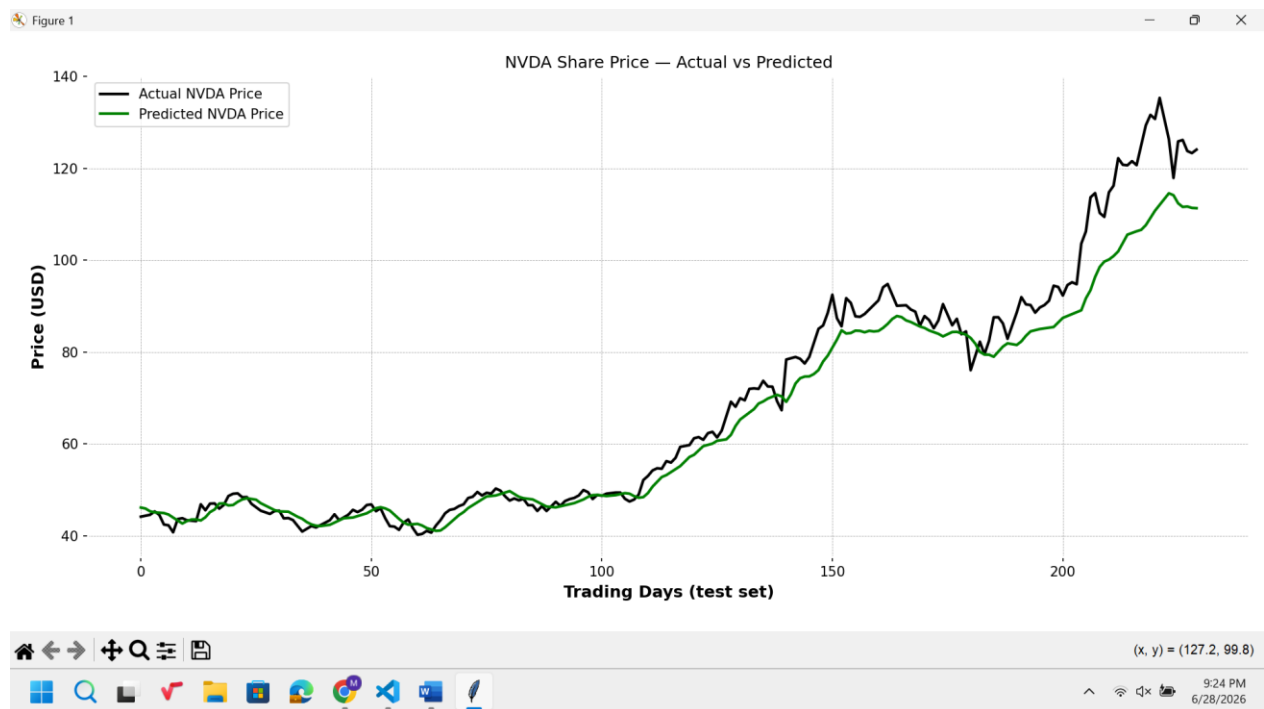
These three cases were conducted on the stock of NVDA data (starting from 2020-01-01 to 2024-07-02), where training was done on the data up until 2023-08-02 while testing was done on the remaining data using GRU.

C.5-1 Multistep (5-day forecast):



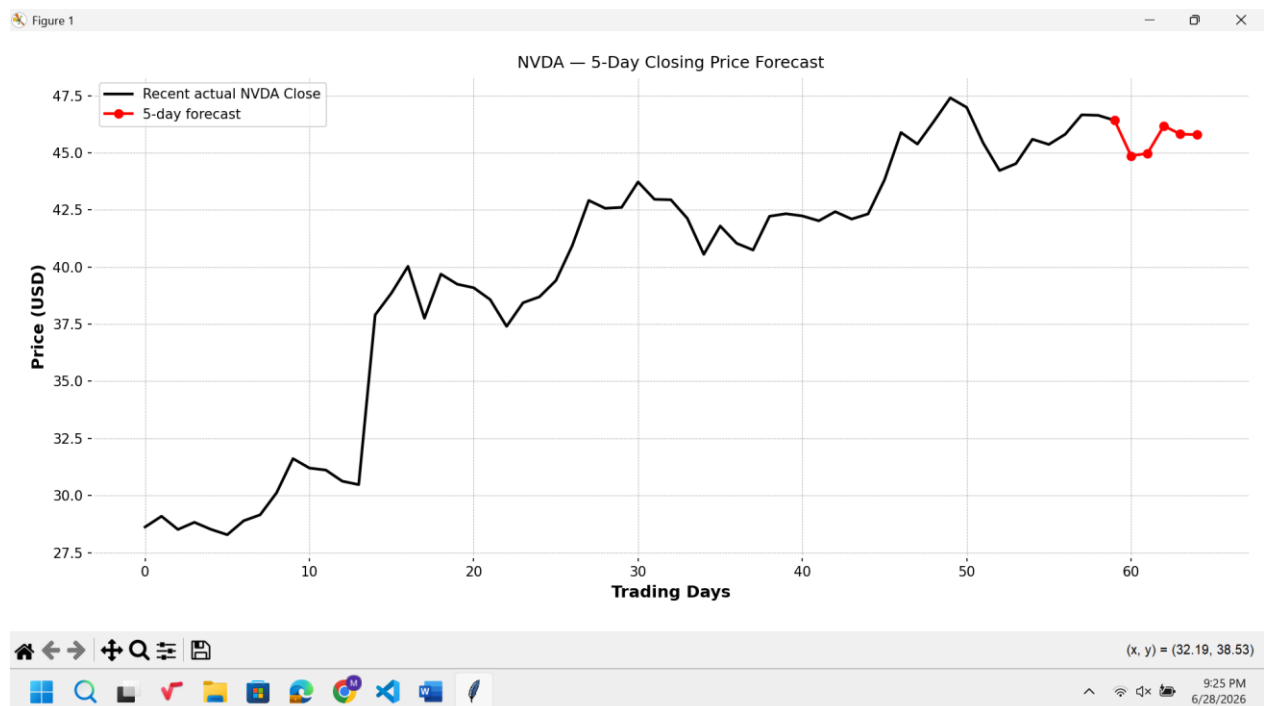
The multivariate forecast for 5 days ahead follows a pattern that is similar to the recent days' price behavior, yet displays higher variance with the passage of time into the future; this is expected. The multivariate model has captured the short-term pattern; however, the model can't predict the unforeseen changes due to news reports.

C.5-2 Multivariate (next-day Close):



The application of the extra variables Open, High, Low, and Volume in addition to the target variable in the multi-input model has played an important role in obtaining a better fit with the closing prices relative to the univariate model in Task C.4. The use of the extra input variables has expanded the horizon of the model for daily events.

C.5-3 Combined (multivariate, 5-day):



The multi-day forecasts obtained from the multi-input model were smoother compared to the multi-step forecasts from the univariate model due to the availability of information through the extra input variables.